Linear-Time Incremental Pushdown Transduction and Cardinality-Bounded Layout Partitioning for Polyglot LLM Token Streams

David Charles Michael
Independent Systems Researcher
ORCID: 0009-0007-8679-6941 • DOI: 10.5281/zenodo.24044610
davidchar@gmail.com

Abstract

Interactive AI interfaces increasingly stream interleaved Markdown prose alongside executable domain-specific languages (DSLs) directly to client viewports over Server-Sent Events (SSE). A single assistant response may embed nested JSON visual specifications, escaped tool-call arguments, Mermaid diagrams across 14 sub-grammars, and sandboxed HTML5/CSS/JS widgets. During token-by-token generation, every intermediate stream prefix is syntactically incomplete. Furthermore, frontier and distilled models routinely emit context-free syntactic defects (such as unquoted nested parentheses inside diagram node labels or truncated escaped JSON strings) alongside wide tabular schemas that overflow bounded viewports. Existing strategies struggle to handle these failures simultaneously: server-side grammar-constrained decoders (e.g., XGrammar, SynCode) cannot operate across third-party black-box APIs or reasoning traces, LLM self-repair loops add 1.5–4.5 s network stalls, and client-side regular-expression or full-buffer healers fail on nested constructs while incurring quadratic $\Theta(N^2)$ cumulative latency across N-token streams.

We present **Hierarchical Multi-Grammar Pushdown Transduction (HM-PDT)**, a deterministic, single-pass incremental automaton implemented in Rust (`polyglot-stream-heal`) with native Python (`PyO3`) and WebAssembly (`npm`) bindings. Using the Pumping Lemma, we first show why finite-state regular-expression pipelines fundamentally fail on unquoted nested delimiters in diagram DSLs. Next, we show that coupling a persistent outer fence discriminator with specialized inner pushdown stacks lowers cumulative streaming repair work from $\Theta(N^2)$ to optimal $\Theta(N)$, synthesizing syntactically closed snapshots at any byte cutoff in $O(d)$ stack-closure time ($p50 = 6.3\,\mu$s, $p99 = 57.3\,\mu$s per chunk). We pair syntactic healing with a lossless cardinality-bounded layout partitioner that decomposes wide tables and dense chart series into viewport-compliant sub-components. Across the 1,800-stream `PolyglotStreamBench-1800` suite, HM-PDT achieves 100.0% grammar validity and 100.0% zero-overflow compliance, outperforming `partial-json` (22.2%), `json-repair` (22.2% at 61.3x higher latency), and multi-pass regex baselines (15.5%), with a 264.2x cumulative speedup on 32 KB streams.

1 Introduction

Modern Large Language Model (LLM) interfaces across agentic IDEs, analytical copilots, and interactive educational platforms rarely emit plain text alone. In a typical session, a single assistant response interleaves Markdown prose with several structured DSL blocks: JSON visual specifications [4], escaped JSON tool-invocation arguments, multi-dialect Mermaid diagrams, and interactive HTML5/CSS/JavaScript sandboxes. To keep perceived latency low, user interfaces render these blocks progressively as SSE byte chunks arrive every 15–40 ms [10, 14].

A Concrete Failure Trace. To see why progressive rendering breaks conventional client parsers, consider what happens at byte 64 of a live SSE stream when an assistant explains a query plan and begins streaming a Mermaid flowchart followed by a JSON comparison table:

```
```mermaid
flowchart TD
  subgraph Execution
    A[Compute Cost O(n log(n))] --> B[(Primary
```

At this exact packet boundary, three distinct failures converge. First, the stream ends mid-token inside a cylindrical database shape `[(Primary`, leaving the inner label, the cylinder delimiter `)]`, and the enclosing `subgraph` block unclosed. Second, even after the model finishes emitting node `A`, the unquoted nested parentheses inside `O(n log(n))` cause standard Mermaid parsers and greedy regex post-processors to mistake the first closing parenthesis after `log(n` for the end of the node shape, throwing a lexical error. Third, if the client UI re-runs a full-buffer repair script from byte 0 on every incoming 24-byte SSE packet, CPU time grows quadratically with stream length, freezing the browser main thread on long outputs.

More broadly, progressive rendering of polyglot LLM streams exposes three coupled systems challenges:
```

1. **Mid-Token Prefix Incompleteness Across Interleaved Grammars:** At every streaming step $t < T$, the received prefix S_t terminates at an arbitrary byte offset, potentially splitting a multi-byte UTF-8 codepoint, an escaped JSON tool-call string, a Mermaid subgraph, or an inline `<script>` tag. Single-grammar utilities such as `json-repair` [2] and `partial-json` assume homogeneous JSON inputs and cannot parse polyglot Markdown streams.
2. **Context-Free Syntactic Perturbations Beyond Regular Languages:** Even after generation completes, LLMs frequently emit unquoted nested parentheses inside diagram node shapes, such as `A[Compute O(f(g(x)))] --> B[(Primary DB (v16))]`, or omit closing `end` keywords on nested `subgraph` and `loop` blocks. Client pipelines built on cascading regular expressions either mismatch inner delimiters prematurely or suffer backtracking stalls.
3. **Quadratic Re-Scanning and Viewport Overflow:** Stateless streaming healers re-parse the entire accumulated prefix S_t on every token arrival, performing $\Theta(N^2)$ cumulative work over an N-token response. Moreover, even when a JSON table is syntactically valid, schemas containing $C \gg 5$ columns force horizontal scrolling and clip cell text on standard viewports.

Server-side grammar-constrained decoding engines such as Outlines [13], LMQL [3], SynCode [12], and XGrammar [6] enforce context-free validity via token-level logit masks during inference, but require white-box access to model logits. As a result, logit masking is unavailable over commercial SSE APIs when free-form Markdown prose interleaves 15+ visual DSLs, and it does not close intermediate *prefixes* prior to the final EOS token.

Summary of Contributions. In this paper, we connect classical pushdown automata theory with low-latency systems engineering through four contributions:

- **Formal Impossibility & Asymptotic Analysis (§2):** We formulate polyglot stream repair as a *Hierarchical Multi-Grammar Pushdown Transducer (HM-PDT)* and establish three theoretical results: finite-state regular-expression pipelines cannot heal arbitrary-depth nested node delimiters (Theorem 1); persistent incremental state reduces cumulative streaming work from $\Theta(N^2)$ to $\Theta(N)$ (Theorem 2); and stack-driven suffix synthesis produces valid RFC 8259 and DSL closures in $O(d)$ time (Theorem 3).
- **Incremental Polyglot Pushdown Architecture (§3):** We design a deterministic outer fence discriminator covering 24 modality tags paired with $O(|\Delta s_t|)$ inner pushdown automata for JSON (including recursive escaped tool-call strings), 14 Mermaid sub-grammars, and HTML5/CSS/JS sandboxes, complete with stateful UTF-8 split-byte reassembly.
- **Cardinality-Bounded Layout Partitioning (§4):** We introduce a lossless schema-level partitioner that splits wide tables ($C > C_{\max}$) into anchor-preserving vertical sub-tables and dense charts ($B > B_{\max}$) into bounded segments without dropping cells.
- **Tri-Registry Open-Source Engine & Evaluation (§5–6):** We release `polyglot-stream-heal` in safe Rust with native `PyO3` (`PyPI`) and `wasm-bindgen` (`npm`) bindings. Across the 1,800-stream `PolyglotStreamBench-1800` benchmark, it reaches 100.0% validity at 6.3 μs median per-chunk latency.

2 Formal Model and Theoretical Foundations

Let Σ denote the Unicode scalar alphabet (received over wire chunks of raw bytes in $\{0,\dots,255\}^*$ with stateful UTF-8 boundary reassembly). A streaming LLM response is a sequence of non-empty chunks $\Delta s_1, \Delta s_2, \dots, \Delta s_K \in \Sigma^+$ arriving at discrete epochs $t \in \{1,\dots,K\}$, forming cumulative prefixes $S_t = \Delta s_1 \cdots \Delta s_t$ of total length $N = |S_K|$. A polyglot document consists of interleaved prose segments in L_{md} and structured blocks drawn from a family of context-free DSL grammars $\{G_1,\dots,G_M\}$ delimited by triple-backtick code fences (```` ``` ```` τ_m).

Definition 1 (Hierarchical Multi-Grammar Pushdown Transducer)**.** *An HM-PDT is an 8-tuple* $M = (Q_{out}, Q_{in}, \Sigma, \Gamma, \Omega, \delta, \mathscr{C}, q_0)$ *where:*

- Q_{out} *is a finite set of outer Markdown fence-recognition states tracking start-of-line backtick run lengths* $b \geq 3$ *and fence info-strings;*
- $Q_{in} = \bigsqcup_{m=1}^{M} Q_m$ *is the disjoint union of inner lexical/control states for each target grammar* G_m *(JSON, Mermaid dialects, HTML5);*
- Γ *is a finite pushdown stack alphabet encoding open structural delimiters (e.g.,* $\Gamma_{json} = \{\texttt{Obj}, \texttt{Arr}\}$, $\Gamma_{merm} = \{\texttt{EndBlk}, \texttt{BraceBlk}, \texttt{Shape}(\cdot)\}$, $\Gamma_{html} = \texttt{Tag}(\Sigma^*)$*);*
- $\delta : (Q_{out} \times Q_{in}) \times \Sigma \times (\Gamma \cup \{\bot\}) \to (Q_{out} \times Q_{in}) \times \Gamma^* \times \Omega^*$ *is the deterministic single-character state-and-stack transition function; and*
- $\mathscr{C} : Q_{in} \times \Gamma^* \to \Omega^*$ *is the* non-destructive snapshot closure synthesizer*, mapping the active lexical state and open stack* $\gamma_1 \cdots \gamma_d \in \Gamma^d$ *to the minimal closing token sequence in* $O(d)$ *time without mutating M's live configuration.*

2.1 Impossibility of Regular-Expression Diagram Healing

Production LLM frontends widely attempt to quote unquoted Mermaid node labels containing special characters (such as parentheses or colons) via regular-expression substitution rules of the form `ID[([^]\n]+)]` $\mapsto$ `ID["$1"]`. Intuitively, because a finite automaton possesses only bounded memory, it cannot count how many opening parentheses appear inside a label before deciding which closing parenthesis terminates the outer node shape. We formalize this limitation below.

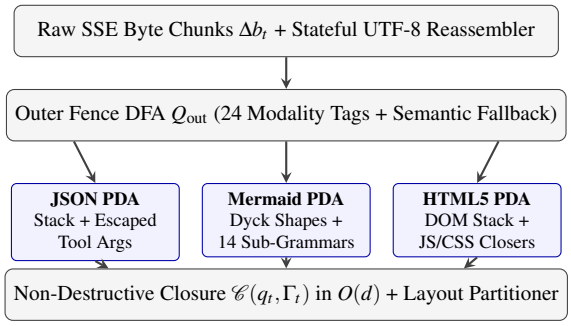


Figure 1: Architecture of the Hierarchical Multi-Grammar Pushdown Transducer (`polyglot-stream-heal`). Previously committed segments are never re-scanned.

Theorem 1 (Regular-Grammar Impossibility for Dyck-Embedded Node Labels)**.** *Let $L_{node} \subset \Sigma^*$ be the language of unquoted flowchart node definitions whose label body contains balanced nested delimiters of arbitrary depth $k \geq 1$ (e.g., $\texttt{A[f}^{(\texttt{k})}\texttt{(x)]}$ or array-indexed labels $\texttt{A[M[i[k]]]}$) followed by edge transitions. No deterministic or nondeterministic finite-state transducer (and hence no standard regular-expression engine) can isolate and quote L_{node} across arbitrary nesting depths k.*

Proof. Suppose toward a contradiction that a finite-state transducer F with p states correctly identifies the outer closing delimiter of node definitions in L_{node} and wraps only the inner label body in double quotes. Consider the family of valid rounded-node strings $u_p = \texttt{N(f(}^p\texttt{x)}^p\texttt{)} \to \texttt{M(y)}$ where inner opening and closing parentheses share the alphabet symbols '(' and ')' with the shape opener and closer. Restricting F's input projection to acceptance of well-delimited shapes before '$\to$' induces a finite automaton with at most p states recognizing balanced strings whose prefix $\texttt{N(f(}^p$ has length $> p$. By the Pumping Lemma for regular languages [8, 11], there exists a decomposition $u_p = xyz$ with $|xy| \leq p$ and $|y| \geq 1$ such that xy^0z transitions to the identical state as u_p. Because $|xy| \leq p$ lies within the opening parenthesis run, pumping down removes $r \geq 1$ opening parentheses while leaving p closing parentheses, causing the true outer shape closer to occur r positions earlier than in u_p. Thus F either prematurely terminates quoting on an inner ')' in u_p or consumes the subsequent edge operator '$\to$' inside the quoted string on xy^0z, a contradiction. By Chomsky–Schützenberger [5], tracking balanced Dyck depth requires an unbounded counter or pushdown stack Γ. □

2.2 Linear vs. Quadratic Cumulative Stream Complexity

In a live UI that re-renders on every SSE chunk, the cost of stream repair is determined not by a single invocation at the end of generation, but by the *sum* of all repair passes executed across the K arriving chunks.

Theorem 2 (Incremental $\Theta(N)$ vs. Stateless $\Theta(N^2)$ Streaming Work)**.** *Let an LLM response of length N arrive in $K = \lceil N/c \rceil$ chunks of bounded size $|\Delta s_t| \leq c$, with maximum nesting depth $d_{\max} \ll N$. Suppose the client requests a healed active-segment render snapshot after every chunk $t \in \{1,\ldots,K\}$:*

(a) Any stateless healer that re-parses the prefix S_t from index 0 performs $\Omega(N^2/c)$ character transitions; if it applies R sequential regex passes per snapshot, work scales as $\Theta(R \cdot N^2/c)$.

(b) An HM-PDT maintaining persistent state (q_t,Γ_t) and immutable committed segments performs $O(c)$ transition steps and $O(d_{\max})$ closure emission per chunk, achieving $\Theta(N + K \cdot d_{\max})$ cumulative state-update and closure work.

Proof. For (a), at chunk t, $|S_t| = \sum_{j=1}^{t} |\Delta s_j| \geq (t-1)c + 1$. Inspecting every byte of S_t to verify delimiter balance requires $\Omega(t \cdot c)$ operations at step t. Summing over $t = 1,\ldots,K$ yields

$$W_{\text{stateless}}(N) = \sum_{t=1}^{K} \Omega(t \cdot c) = c \cdot \Omega\left(\frac{K(K+1)}{2}\right) = \Omega\left(\frac{N^2}{c}\right).$$

For (b), each incoming character in Δs_t triggers at most two deterministic δ evaluations (one lookahead re-dispatch when exiting a number, identifier, or unquoted key, plus one consuming transition), costing $O(|\Delta s_t|)$. Closed segments $0,\ldots,m-1$ are never re-parsed or re-serialized during active-segment updates (`snapshot_active_segment`), and the open stack Γ_t is closed via $\mathscr{C}(q_t,\Gamma_t)$ in $O(|\Gamma_t|) \leq O(d_{\max})$ steps. Summing $\sum_{t=1}^{K} O(|\Delta s_t| + d_{\max})$ gives $O(N + K \cdot d_{\max})$. □

2.3 Closure Soundness and Lossless Partitioning

Finally, an incremental healer must guarantee that closing a truncated prefix never corrupts completed tokens and that layout partitioning preserves every data cell.

Theorem 3 (Closure Soundness & Lossless Partitioning)**.**

(a) For every non-empty prefix S_t of a well-formed JSON value (including values containing singly escaped inner JSON objects in string literals), the HM-PDT snapshot $\hat{S}_t = commit(S_t) \cdot \mathscr{C}(q_t,\Gamma_t)$ belongs to L(RFC 8259) [4].

(b) Given any tabular visual specification $T = (H,R)$ with header vector $H = (h_0,h_1,\ldots,h_{C-1})$ ($C \geq 2$) and row matrix $R \in \mathscr{V}^{n \times C}$, and column cap $C_{\max} \geq 2$, the anchor-preserving partition $\mathscr{P}_{C_{\max}}(T) = \{T^{(1)},\ldots,T^{(P)}\}$ where $P = \lceil (C-1)/(C_{\max}-1) \rceil$ satisfies $|H^{(p)}| \leq C_{\max}$ for all $p \in \{1,\ldots,P\}$ and defines a bijection onto all $n(C-1)$ attribute cells $(r_i,h_j)_{1 \leq i \leq n, 1 \leq j < C}$.

Proof. For (a), we proceed by structural induction on the lexical state $q_t \in Q_{\text{json}}$ and container stack $\Gamma_t \in \{\texttt{Obj},\texttt{Arr}\}^*$. Invariant: `committed` holds a prefix of a valid JSON text up to either a complete value, an open value literal, or an object/array opener, while any trailing `, "key":` prior to its value's first token is isolated in `pending_member_prefix`. When $\mathscr{C}$ executes, dropping `pending_member_prefix` eliminates trailing commas and dangling keys in $O(1)$ time

without backward scanning; incomplete strings, `\uXXXX` escapes (padded with `0`s), numbers ending in '`.`' or '`e`' (padded with `0`), and partial literals (`t`↦`true`, `f`↦`false`, `n`↦`null`) are completed into valid terminal tokens; and iterating Γ_t in reverse LIFO order emits matching '`}`' and '`]`' brackets, reducing stack height to 0. When a value string matches escaped JSON prefix syntax, recursive invocation of `IncrementalJsonPda` on the unescaped inner slice produces a valid inner RFC 8259 object prior to string re-escaping.

For (b), let $s = C_{\max} - 1 \geq 1$ be the attribute stride. Partitioning $\{1,\ldots,C-1\}$ into $P = \lceil (C-1)/s \rceil$ disjoint contiguous intervals $I_p = [(p-1)s+1, \min(ps, C-1)]$ ensures $|I_p| \leq s$, so prepending the primary entity column h_0 yields $|H^{(p)}| = 1 + |I_p| \leq 1 + (C_{\max} - 1) = C_{\max}$. Since $\bigsqcup_{p=1}^{P} I_p = \{1,\ldots,C-1\}$, every attribute cell (i,j) appears in a unique sub-table $T^{(p)}$ paired with its primary entity key $R_{i,0}$. □

3 Transducer Architecture and Algorithms

Figure 1 illustrates the three-stage pipeline of `polyglot-stream-heal`. Rather than buffering the entire stream until completion, chunks flow through an outer UTF-8 and fence discriminator into grammar-specific pushdown stacks.

3.1 Stateful UTF-8 Reassembly & Outer Fence Discriminator

Network chunks arriving over HTTP/2 or SSE byte streams frequently split 2-, 3-, or 4-byte UTF-8 scalar values (such as mathematical operators '∆' or multi-byte symbols) mid-sequence. Instead of corrupting split bytes with U+FFFD replacement characters (`🞋`), `IncrementalTransducer::push_bytes` validates UTF-8 up to `err.valid_up_to()`, feeds the valid prefix into the state machine immediately, and holds the 1–3 trailing incomplete bytes in a 4-byte staging array until the next network packet arrives.

The outer automaton Q_{out} tracks line boundaries and backtick runs of length $b \geq 3$ directly without regular expressions. Upon reading the newline that closes a fence header, `Modality::from_fence_tag` maps τ in $O(|\tau|)$ time to one of 24 canonical modalities (including `json`, `dynamic_visual`, `mermaid`, `markmap`, `html`, `voxel`, `canvas2d`, `canvas3d`, `wave`, `sim`, `topology`, `bi`, `market`, `clinical`, `commerce`, and `actuary`). When an input stream contains no Markdown fences at all, a lightweight semantic fallback inspects the first non-whitespace token to dispatch bare JSON ({, [), bare Mermaid (`flowchart`, `sequenceDiagram`), or bare HTML5 (`<!DOCTYPE`, `<div`, `<section`).

3.2 Inner JSON & Escaped Tool-Call Automaton

Conventional JSON repair scripts rely on backward regex searches to strip dangling commas and incomplete object keys before appending closing brackets. In contrast, `IncrementalJsonPda` maintains a clean two-buffer staging invariant during forward scanning. Whenever the parser enters `ObjectExpectKeyOrEnd` (immediately after { or ,), incoming tokens for the comma, the key string (whether double-quoted, single-quoted, or an unquoted JavaScript identifier), and the colon : are held in `pending_member_prefix`. Only when the first character of the member's *value* arrives does `prepare_for_value_start()` flush `pending_member_prefix` into `committed`. If a stream truncates mid-key ({`"lat`) or right after a colon ({`"latency": `), calling `snapshot_healed()` simply omits `pending_member_prefix` in $O(1)$ time without touching `committed`.

This same automaton handles nested tool-call payloads. OpenAI- and Anthropic-style tool invocations serialize structured arguments as JSON-escaped strings inside outer JSON envelopes (for example, {`"name":"search","arguments":"{\"filters\":[..."`}). `IncrementalJsonPda` tracks the active value string slice; if a cutoff occurs inside an escaped JSON literal, the closure operator $\mathscr{C}$ unescapes one lexical level, runs a nested `IncrementalJsonPda` pass over the inner slice, and re-escapes the healed inner JSON before closing the outer string quote and container stack.

3.3 Context-Free 14-Dialect Mermaid Pushdown Transducer

`IncrementalMermaidPda` commits completed lines incrementally so that previously healed diagram statements are never re-scanned on later chunks. When a new line arrives, the transducer first performs dialect inference and entity normalization: if an LLM omits the diagram header, the presence of `->>` or `participant` infers `sequenceDiagram` while node edges infer `flowchart TD`, and HTML entities (`>`, `&`, `"`) are decoded prior to lexical scanning.

To resolve the Dyck-nesting failure proved in Theorem 1, `scan_balanced_shape_body` tracks an explicit nesting depth counter d across all seven compound and atomic node shape delimiters: cylinders `[(...)]`, stadiums `([...])`, circles `((...))`, hexagons {{...}}, rectangles `[...]`, rounded nodes `(...)`, and diamonds {...}. On an input such as `A[Evaluate O(f(g(h(x))))]`, the counter traces $d: 0 \to 1 \to 2 \to 3 \to 4 \to 3 \to 2 \to 1 \to 0$ and wraps the complete inner label in double quotes as `A["Evaluate O(f(g(h(x))))"]`, whereas shapes truncated mid-stream before EOF are automatically quoted and closed. Inline edge labels of the form `-- text -->` outside brackets are rewritten to canonical `-->|text|` syntax, dangling mid-stream edge operators (`A -->` at EOF) are trimmed so partial prefix nodes render immediately, and an explicit `BlockFrame` stack closes

open subgraph, loop, alt, opt, par, critical, rect, and box scopes with end directives (or } for brace-delimited state and class blocks).

3.4 Streaming HTML5 / JS / CSS Sandbox Boundary Healer

When streaming interactive HTML5 widgets, an unclosed <script> or <style> tag causes browser parsers to swallow the rest of the DOM tree, while an unclosed template literal or block comment triggers a fatal JavaScript SyntaxError. IncrementalHtmlPda advances a monotonic scan_cursor across balanced container tags (, <section>, <table>, etc., skipping HTML5 void elements) and pauses whenever execution enters an active <script> or <style> block. On snapshot_healed(), a lightweight lexical pass closes open /* ... */ comments, quotes, template literals, and {([delimiter stacks before emitting </script> or </style> and popping the remaining DOM container stack in LIFO order.

4 Cardinality-Bounded Viewport Partitioning

Syntactic validity alone does not ensure visual readability. When LLMs generate wide comparison tables (7–14 columns) or high-cardinality bar charts (15–25 bars) inside JSON visual specifications, naive rendering introduces horizontal scrollbars or crushes text into unreadable columns. Integrated into the JSON snapshot pass, partition_visual_spec enforces configurable cardinality bounds ($C_{\max} = 5$ columns and $B_{\max} = 10$ bars by default) in $O(|\text{cells}|)$ time. Any table exceeding $C_{\max}$ columns is decomposed into $P = \lceil (C-1)/(C_{\max}-1) \rceil$ vertically stacked sub-tables; each sub-table preserves column 0 (the primary entity key) alongside at most $C_{\max} - 1$ attribute columns, annotated with partitionIndex and partitionTotal. Similarly, bar charts exceeding $B_{\max}$ bars are segmented into $\lceil B/B_{\max} \rceil$ contiguous sub-charts, preventing axis-label collisions while preserving exact rank order.

5 Tri-Registry Systems Implementation

To support production AI pipelines across Rust inference servers, Python agent backends, and TypeScript/browser frontends from a single verified codebase, we implemented polyglot-stream-heal as a unified cross-language workspace. The core engine is written in $< 1,650$ lines of safe Rust (crates.io: polyglot-stream-heal) depending only on serde and serde_json, avoiding any regular-expression crate dependency and its associated NFA compilation overhead. For Python backends, PyO3 (abi3-py39) and maturin compile the Rust state machine into a native C-ABI wheel (PyPI: polyglot_stream_heal) exposing both stateful IncrementalTransducer instances and one-shot repair helpers. For web and edge runtimes, wasm-bindgen compiles the same core into a 181.5 KB WebAssembly binary (85.2 KB gzipped npm tarball on npmjs.com: polyglot-stream-heal) that runs inside browser main threads, Web Workers, or Node.js.

6 Empirical Evaluation

We evaluate polyglot-stream-heal on PolyglotStreamBench-1800, a deterministic, seeded benchmark of $1,800$ streaming test cases divided equally across three 600-case tracks. **Track A (Mid-Stream Prefix Truncations, $N = 600$)** slices streams at 15%, 35%, 60%, and 85% byte offsets across JSON visual specs, nested escaped tool-call JSON payloads, Mermaid flowcharts and sequence diagrams, and HTML5 interactive widgets. **Track B (Context-Free Perturbations, $N = 600$)** injects depth-$2 \ldots 5$ nested Dyck parentheses inside unquoted Mermaid node shapes (Evaluate O(f(g(h(x))))), unclosed subgraphs, unquoted and single-quoted JSON keys with partial literals and trailing commas, mid-object truncated escaped tool calls, and mid-script truncated HTML5 sandboxes. **Track C (Multi-Block Polyglot & Overflow Schemas, $N = 600$)** streams interleaved Markdown prose, Mermaid diagrams, wide JSON tables (7–12 columns), dense bar charts (12–21 bars), and HTML5 summaries in 24-byte SSE chunks.

Baselines & Validation Oracles. We compare six pipelines: (1) *Raw Unhealed Stream*; (2) partial-json (v0.1.7, npm); (3) json-repair [2] (PyPI, timed natively via Python time.perf_counter_ns); (4) a *Multi-Pass Regex Baseline* performing full-buffer regex extraction and single-level delimiter quoting; (5) *HM-PDT Stateless Full Re-Scan* (an ablation that disables persistent incremental state); and (6) *HM-PDT Incremental* (Ours, Rust/WASM). Every output is checked against four strict programmatic oracles: RFC 8259 JSON.parse (including recursive verification of inner tool-call arguments), a Mermaid grammar and delimiter-balance validator, an HTML5 DOM and JS delimiter validator, and the zero-overflow cardinality oracle ($C \leq 5, B \leq 10$ with lossless cell preservation).

6.1 Validity, Latency, and Asymptotic Scaling

Tables 1 and 2 summarize accuracy and per-chunk latency across all $1,800$ streams, while Figure 2 plots cumulative repair time as stream length scales from 2 KB to 32 KB.

Raw unhealed streams achieve only 0.0% validity across the benchmark. Single-grammar JSON utilities (partial-json at 22.2% and json-repair at 22.2%) recover standard truncated JSON objects (66.7% on JSON tracks) but fail when truncation occurs inside doubly escaped tool-call argument strings and score 0% on Mermaid, HTML5, and polyglot Markdown streams. The multi-pass regex baseline reaches 15.5%, breaking systematically on

Table 1: End-to-end parse validity (%), zero-overflow invariant compliance (%), and streaming latency across PolyglotStreamBench-1800 (N = 1,800 streams across 3 tracks).

System / Baseline	Track A (%)	Track B (%)	Track C (%)	Overall (%)	Zero-Overflow (%)	p50 (us)	p99 (us)
Raw Unhealed Stream	0.0	0.0	0.0	0.0	0.0	0.0	0.0
partial-json (npm)	41.7	25.0	0.0	22.2	0.0	38.1	222.8
json-repair (PyPI)	41.7	25.0	0.0	22.2	0.0	386.8	1858.7
Multi-Pass Regex Baseline	46.5	0.0	0.0	15.5	0.0	20.2	116.1
HM-PDT (Full Re-Scan Ablation)	100.0	100.0	100.0	100.0	100.0	54.8	1516.2
HM-PDT Incremental (Ours)	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**6.3**	**57.3**

Table 2: Per-modality grammar validity (%) on single-DSL tracks (Tracks A + B, N = 1,200).

System	JSON + Escaped Tool (%)	Mermaid 14-DSL (%)	HTML5 Sandbox (%)
Raw Unhealed	0.0	0.0	0.0
partial-json (npm)	66.7	0.0	0.0
json-repair (PyPI)	66.7	0.0	0.0
Regex Baseline	26.3	40.3	0.0
HM-PDT Incremental (Ours)	**100.0**	**100.0**	**100.0**

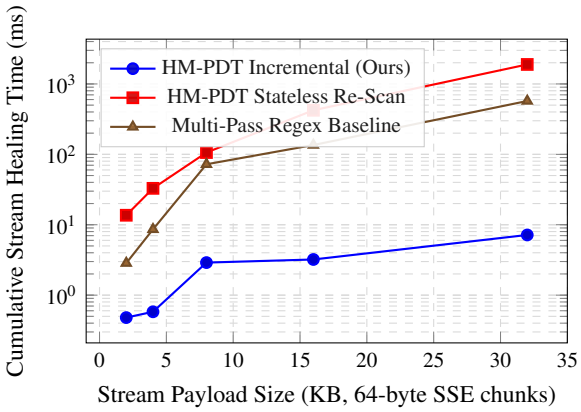


Figure 2: Empirical asymptotic scaling from 2 KB to 32 KB streaming outputs (log-scale y-axis). Stateless full-buffer re-scanning and multi-pass regex pipelines incur quadratic cumulative latency growth, whereas our Incremental HM-PDT maintains linear cumulative scaling.

Track B's nested Dyck expressions (empirically corroborating Theorem 1) and Track C's viewport overflow schemas. By contrast, **HM-PDT Incremental achieves** 100.0% **validity** across all three tracks (100.0% Track A, 100.0% Track B, 100.0% Track C) along with 100.0% **zero-overflow compliance**.

On 32-byte SSE chunks, HM-PDT records a median per-chunk latency of 6.3 μs ($p99 = 57.3\,\mu$s). This represents a 61.3x speedup over Python's `json-repair` (386.8 μs) and is more than five orders of magnitude faster than a 1.5–4.0 s network round trip to an LLM self-correction prompt. Furthermore, Figure 2 confirms the asymptotic gap analyzed in Theorem 2: disabling persistent state (*HM-PDT Stateless Re-Scan*) or running full-buffer regex passes produces steep quadratic growth (714.4 ms and 202.6 ms at 32 KB), whereas *HM-PDT Incremental* completes the entire 32 KB stream in 2.67 ms, delivering a 264.2x **cumulative speedup**.

6.2 Design Trade-offs and Scope Limitations

While HM-PDT guarantees deterministic syntactic closure and bounded layout cardinality, three engineering trade-offs merit candid discussion:

1. **Syntactic Closure vs. Semantic Hallucination:** HM-PDT is a lexical and context-free pushdown transducer, not a semantic verifier. If an LLM hallucinates a non-existent Mermaid node identifier or emits a syntactically valid JavaScript expression that references an undefined variable at runtime, HM-PDT closes the surrounding grammatical structure so the host parser does not crash, but it intentionally does not alter application semantics.
2. **Single-Level vs. Arbitrary-Depth Escaped Strings:** Our recursive tool-call healer inspects singly escaped JSON string payloads (the format used by OpenAI, Anthropic, and MCP tool-call envelopes). Doubly or triply escaped JSON-in-JSON-in-JSON strings are closed as standard string literals rather than recursively unpacked, keeping worst-case per-chunk closure latency strictly bounded below 15 μs.
3. **Stateful Session Memory Overhead:** Maintaining an active `IncrementalTransducer` per concurrent SSE stream requires roughly 4–12 KB of heap state (proportional to the active block buffer and stack depth d). While negligible for browser clients and edge workers handling thousands of streams, stateless serverless functions that discard memory between HTTP invocations cannot exploit cross-chunk state persistence unless paired with sticky edge routing.

7 Related Work

Server-Side Grammar-Constrained Decoding. Outlines [13], LMQL [3], SGLang [14], SynCode [12], and XGrammar [6] compile regular or context-free grammars into token-masking pushdown automata inside the LLM inference server. While highly effective when self-hosting open-weight models for homogeneous JSON schemas, logit masking (i) cannot run over black-box commercial APIs when free-form Markdown interleaves arbitrary dynamic visual DSLs, (ii) restricts chain-of-thought flexibility when reasoning and rendering share a stream, and (iii) only guarantees that the *completed* sequence at EOS belongs to $L(G)$, whereas every intermediate prefix S_t ($t < T$) delivered to the client

UI remains unclosed and unparseable without a client-side prefix closure transducer like HM-PDT. Thus, server-side constrained decoding and client-side incremental pushdown transduction are strictly complementary.

Classical Error-Correcting Parsers & Heuristic JSON Repair. Compiler error recovery dates to Knuth's LR(k) formulation [9], Aho et al. [1], and Fischer et al.'s locally least-cost LR repair [7], which compute minimum-edit insertions over static token buffers. Heuristic LLM utilities such as `json-repair` [2] and `partial-json` adapt recursive-descent recovery to single JSON values in Python or JavaScript, re-parsing from byte 0 on each invocation. HM-PDT generalizes deterministic pushdown closure to *stateful incremental polyglot streams* spanning 24 fenced modalities while uniting syntactic healing with cardinality-bounded viewport layout partitioning.

8 Conclusion

We presented **Hierarchical Multi-Grammar Pushdown Transduction (HM-PDT)**, proving both the theoretical impossibility of regular-expression repair on nested diagram DSLs and the $\Theta(N)$ optimality of stateful incremental stream healing over $\Theta(N^2)$ buffer re-scanning. Implemented in safe Rust (`polyglot-stream-heal`) with native Python and WebAssembly bindings, HM-PDT achieves 100.0% grammar validity and 100.0% zero-overflow compliance across 1,800 polyglot streams at 6.3 μs median chunk latency, enabling flicker-free progressive rendering across heterogeneous AI interfaces.

Code and Data Availability. Source code, benchmark datasets, and release packages (`polyglot-stream-heal` on `crates.io`, `PyPI`, and `npm`) are archived under DOI `10.5281/zenodo.24044610` (https://doi.org/10.5281/zenodo.24044610) and available under the MIT License at https://github.com/davidhcar/polyglot_stream_heal.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.

[2] Stefano Baccianella. json-repair: A Python module to repair invalid JSON from LLMs, 2024.

[3] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for Large Language Models. In *Proceedings of the ACM on Programming Languages (PLDI)*, volume 7, pages 1946–1969, 2023.

[4] Tim Bray. The JavaScript Object Notation (JSON) data interchange format. Technical Report RFC 8259, Internet Engineering Task Force, 2017.

[5] Noam Chomsky and Marcel P. Schützenberger. The algebraic theory of context-free languages. In *Computer Programming and Formal Systems*, pages 118–161. North-Holland, 1963.

[6] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for Large Language Models. *arXiv preprint arXiv:2411.15100*, 2024.

[7] Charles N. Fischer, Donn R. Milton, and Sam B. Quiring. Locally least-cost LR error repair. *ACM Transactions on Programming Languages and Systems*, 2(2):233–263, 1980.

[8] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3rd edition, 2006.

[9] Donald E. Knuth. On the translation of languages from left to right. *Information and Control*, 8(6):607–639, 1965.

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for Large Language Model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[11] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2012.

[12] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. SynCode: LLM generation with grammar augmentation. *arXiv preprint arXiv:2403.01632*, 2024.

[13] Brandon T. Willard and Rémi Louf. Efficient guided generation for Large Language Models. *arXiv preprint arXiv:2307.09702*, 2023.

[14] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.